

ok. restart guide to newbee. I will keep it as permanent setup pdf.

물론입니다!

아래는 *****"Ubuntu22 · C++17 · Qt DBus 기반 BLE 5.x 광고 송출 시스템 신규 설치부터 실제 BLE 광고 송출까지, BLE/D-Bus 완전 초보도 성공하는 방식"***을 최소 장애·최대 신뢰 기준으로 다시 작성한 "PERMANENT GUIDE"입니다.**

(BLE 광고/D-Bus 광고 오브젝트 구조, 시스템 환경, 실시간 검증까지 한 번에!)

BLE 광고 (robot_ble_communicator) 영구 설치/구동 가이드

- Newbie/실전 엔지니어 장기 참고용 -

1. 시스템 준비

- 운영체제: Ubuntu 22.04 (혹은 20.04이상)
- BLE 동글/어댑터: BLE 5.x 지원 필요 (hci0로 인식)
- 필수 패키지:

```
sudo apt update
sudo apt install build-essential qtbase5-dev qttools5-dev qt5-qmake libqt5dbus5 qt5-c
git bluez bluez-tools dbus libdbus-1-dev
```

- (권장) BLE Dongle 확인:

```
sudo hciconfig
sudo hciconfig hci0 up
```

2. Qt 프로젝트 빌드

1. 소스 다운로드/복사

- (github clone 등)

2. 빌드

```
cd ~/ws/robot_ble_communicator
mkdir build
cd build
qmake ..
make -j4
```

3. BLE 광고 코드 작성/점검 - 핵심 구조 참고

- 광고 오브젝트는 반드시 **DBus system bus + org.bluez.LEAdvertisement1 인터페이스**로 등록
- Qt 예제(최소):

```
// BLE 광고 오브젝트 (system bus에 반드시 등록)
class QtBleAdvertisement : public QDBusAbstractAdaptor {
    Q_OBJECT
    Q_CLASSINFO("D-Bus Interface", "org.bluez.LEAdvertisement1")
    Q_PROPERTY(QString Type READ type)
    Q_PROPERTY(QVariantMap ManufacturerData READ manufacturerData NOTIFY manufacturerDataChanged)
public:
    explicit QtBleAdvertisement(QObject* parent, const QByteArray& advData) : QDBusAbstractAdaptor(parent, "org.bluez.LEAdvertisement1", 1) {}
    QString type() const { return "peripheral"; }
    QVariantMap manufacturerData() const {
        QVariantMap m;
        m.insert(QString::number(0xFFFF), m_advData); // FFFF는 제조사 ID
        return m;
    }
    void setAdvData(const QByteArray& adv) {
        if (m_advData != adv) {
            m_advData = adv;
            emit manufacturerDataChanged();
        }
    }
signals:
    void manufacturerDataChanged();
private:
    QByteArray m_advData;
};
```

4. 실행 환경 정석화 (systemd 서비스 등록 필수!)

(A) systemd 서비스 파일 작성

- /etc/systemd/system/robot_ble_communicator.service

```
[Unit]
Description=Robot BLE Communicator Service
After=bluetooth.target

[Service]
ExecStart=/path/to/robot_ble_communicator
User=root
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

- 파일 저장 후 다음을 실행:

```
sudo systemctl daemon-reload
sudo systemctl enable robot_ble_communicator
```

```
sudo systemctl start robot_ble_communicator
sudo systemctl status robot_ble_communicator
```

(B) 서비스가 system bus에 광고 오브젝트 등록

- 반드시 root/systemd로 기동해야 bluez가 보는 system bus에 export

5. 실제 광고 오브젝트(system bus) 등록 확인

1. busctl로 광고 오브젝트 확인

```
sudo busctl tree org.bluez
```

→ /org/robot/adv12345 등 오브젝트가 나와야 함

2. 광고 D-Bus 오브젝트 introspect (내용 보기)

```
sudo busctl introspect org.bluez /org/robot/adv12345
```

→ ManufacturerData, Type 등 property/메서드 확인

6. 실시간 BLE 광고 송출 데이터 변경 체크

1. dbus-monitor로 property 변화 감시

```
sudo dbus-monitor --system "path=/org/robot/adv12345"
```

→ [PropertiesChanged] 시그널에 ManufacturerData 등 광고값 entry 변화 나오면 OK!

2. (옵션) BLE 스캐너/스니퍼/스마트폰 앱에서 실제커스텀 광고가 수신되는지 확인

7. 코딩 가이드 요약 체크리스트

- [x] QtBleAdvertisement: org.bluez.LEAdvertisement1로 QDBusAbstractAdaptor + Q_PROPERTY + Q_SIGNAL(Notify) 완비
- [x] QVariantMap직렬화 키: QString::number(0xNNNN), 값: QByteArray(배열)
- [x] system bus(QDBusConnection::systemBus()) export, root 권한 혹은 systemd 서비스로 실행
- [x] RegisterAdvertisement 전, 광고 데이터(ManufacturerData) 정상값 세팅
- [x] busctl/dbus-monitor에서 등록 및 신호 확인 필수

8. 문제 발생시 체크 순서

1. busctl tree org.bluez에서 오브젝트가 보이지 않으면 → system bus 등록, 권한, 서비스 실행 환경 점검
2. 오브젝트만 보이고 [PropertiesChanged]가 안 뜨면 → Q_PROPERTY/Notify/emit 구조 점검
3. bluez 광고 등록 실패/에러로그(ManufacturerData 등) → DBus property 포맷/QVariantMap 직렬화 점검
4. 디버거시 qDebug() + BLE 스니퍼로 실제 광고값 비교, 필요시 example-advertisement.py와 동작 비교

부록: FAQ

- Q: 그냥 `sudo ./robot_ble_communicator` 해도 되나요?
 - 불가피할 때만! 하지만 `systemd` 서비스로 등록하면 실제 `system bus`에 완벽하게 align(최고 권장)
- Q: `busctl`에서 오브젝트 안 보이면?
 - `systemd` 서비스 실행, 권한, `system bus export` 실행 필요
- Q: BLE 광고값/형식 문제?
 - 반드시 `ManufacturerData`는 `dict<uint16, byte array>`의 DBus 포맷!
- Q: 로컬 일반실행으로 정상처럼 보이지만 광고 송출 안 됨?
 - 거의 항상 `system bus` 미등록 or `environment mismatch`

최종 결론(실전 핵심)

- "`busctl tree org.bluez`"로 BLE 광고 오브젝트가 `system bus(bluez가 보는 D-Bus)`에 보이는 것, 이것이 진정한 'BLE 광고 시작'임!
- BLE 시스템은 "코드 인상"이 아니라, "등록 결과(`systemd` 서비스, `busctl` 등) 실제 확인"이 99% 성공의 열쇠!
- 실 BLE 광고 수신까지, 이 프로세스만 재현하면 시스템 리부팅해도 항상 로봇 BLE 광고 송출 시스템 완성!

이 가이드 PDF로 저장, 향후 신규 환경 세팅·장애 진단·운영까지 "실전 현장 매뉴얼"로 활용 바랍니다.

(시스템 문제 생기면 `busctl/dbus-monitor`/블루투스 로그부터 확인하세요!)

추가로:

- `systemd` 서비스용 템플릿 파일,
- 프로젝트별 코드 구조 / 광고 데이터 표준 설계 문서(한글/영문 PDF)
필요하시면 언제든지 말씀해 주세요!